Gabriel Martinez
WES 237C
Project 2

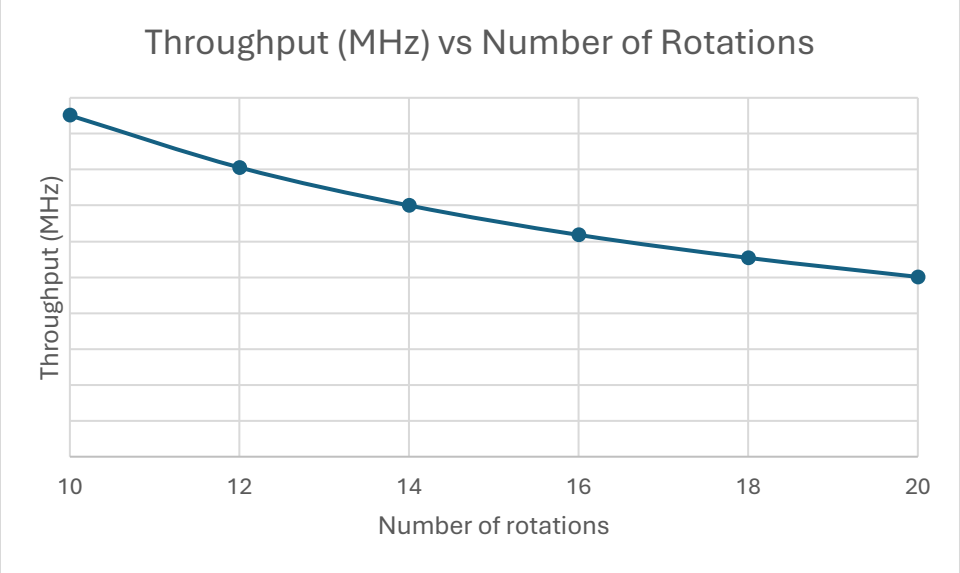
Project 2: CORDIC

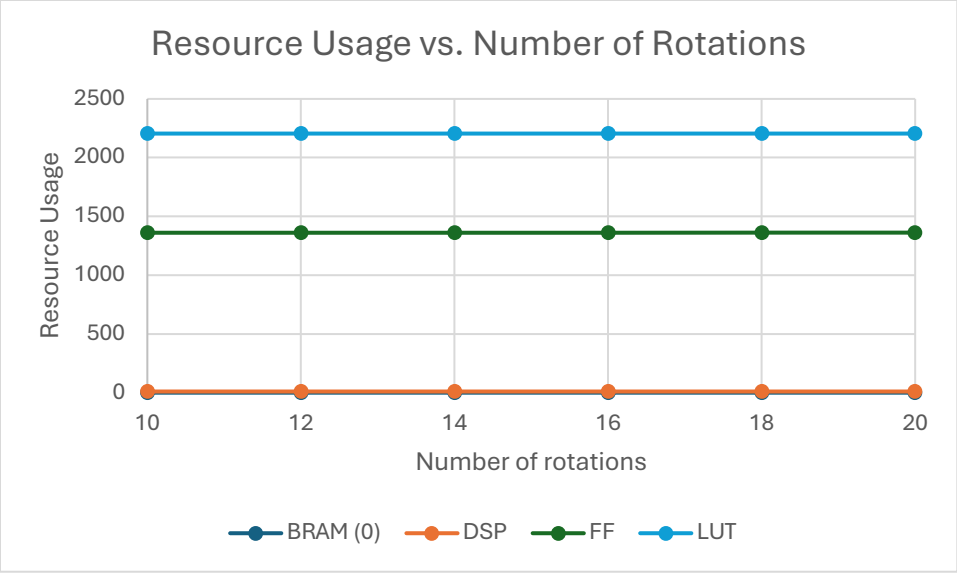
Question 1.

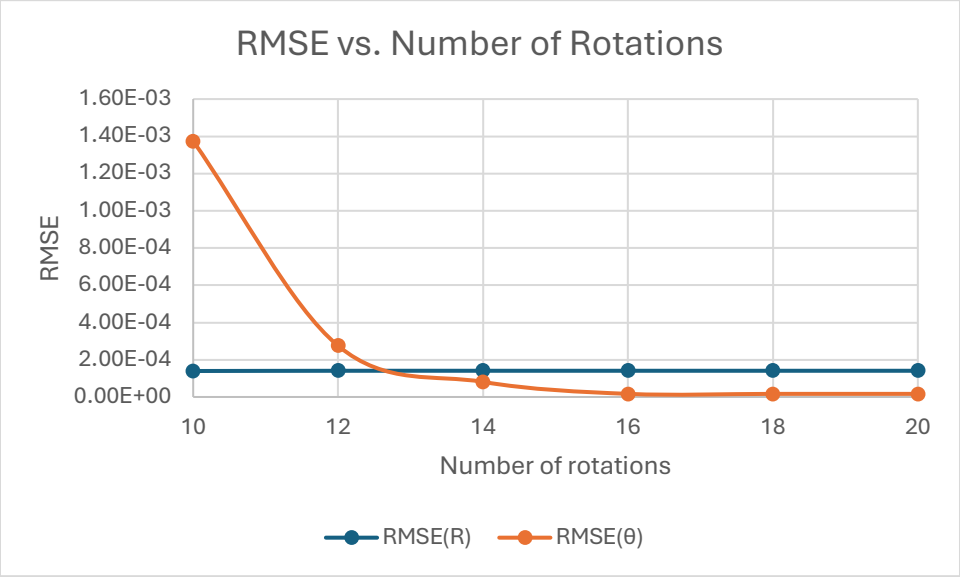
1a)

Rotations	Latency (cycles)	II	Throughput (Hz)	BRAM	DSP	FF	LUT	RMSE(R)	RMSE(θ)
10	144	145	.951	0	10	1361	2205	1.39518e-4	1.37389e-3
12	170	171	.806	0	10	1361	2205	1.40709e-4	2.74922e-4
14	196	197	.700	0	10	1361	2205	1.40761e-4	8.09574e-5
16	222	223	.618	0	10	1361	2205	1.40761e-4	1.66648e-5
18	248	249	.554	0	10	1362	2205	1.40761e-4	1.66648e-5
20	274	275	.501	0	10	1362	2205	1.40761e-4	1.66648e-5

1b)







1c) After increasing the number of rotations beyond 16, no further improvement was observed in the RMSE values.

Question 2.

Since ap_fixed is a signed data type, one bit is used for the sign. The remaining integer bits must be large enough to represent the variable's range.

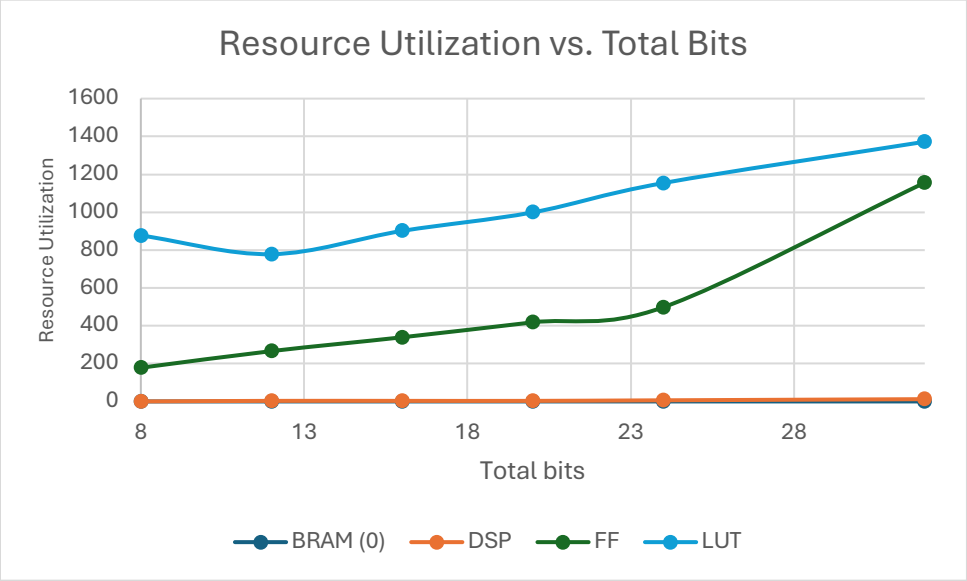
- x and y: 2 bits since both range from -1 to 1, so we need 1 sign bit and 1 integer bit.
- r: 2 bits since the maximum radius is $\sqrt{2} \approx 1.414$, so we need 1 sign bit and 1 integer bit.

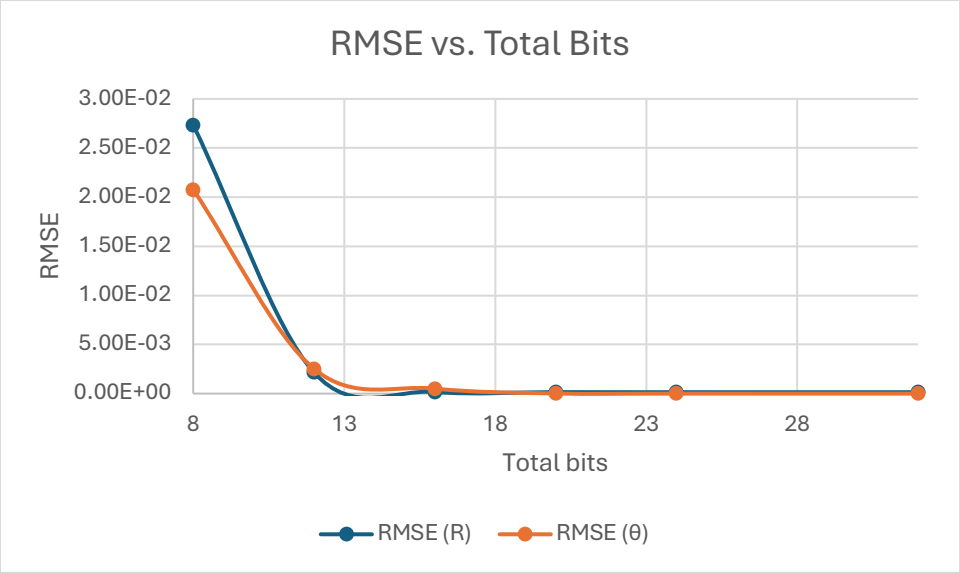
- θ: 3 bits since the angle ranges from $-\pi$ to π (approximately -3.14 to 3.14), so we need 1 sign bit and 2 integer bits.

Answer: x, y, and r require 2 integer bits each, while θ requires 3 bits. Therefore, 3 integer bits will be used for all variables in Question 2(b).

2b)

Total bits (W)	Estimated clock (ns)	Latency (cycles)	II (cycles)	Throughput (MHz)	BRAM	DSP	FF	LUT	RMSE (R)	RMSE (θ)
8	7.274	46	47	2.925	0	0	178	877	2.735e-02	2.078e-02
12	7.280	47	48	2.862	0	3	266	778	2.153e-03	2.524e-03
16	7.229	47	48	2.882	0	3	339	901	1.712e-04	5.095e-04
20	6.554	47	48	3.179	0	3	418	999	1.495e-04	1.375e-05
24	6.554	47	48	3.179	0	6	498	1154	1.414e-04	1.161e-06
32	7.184	68	69	2.017	0	12	1156	1372	1.407e-04	6.172e-07

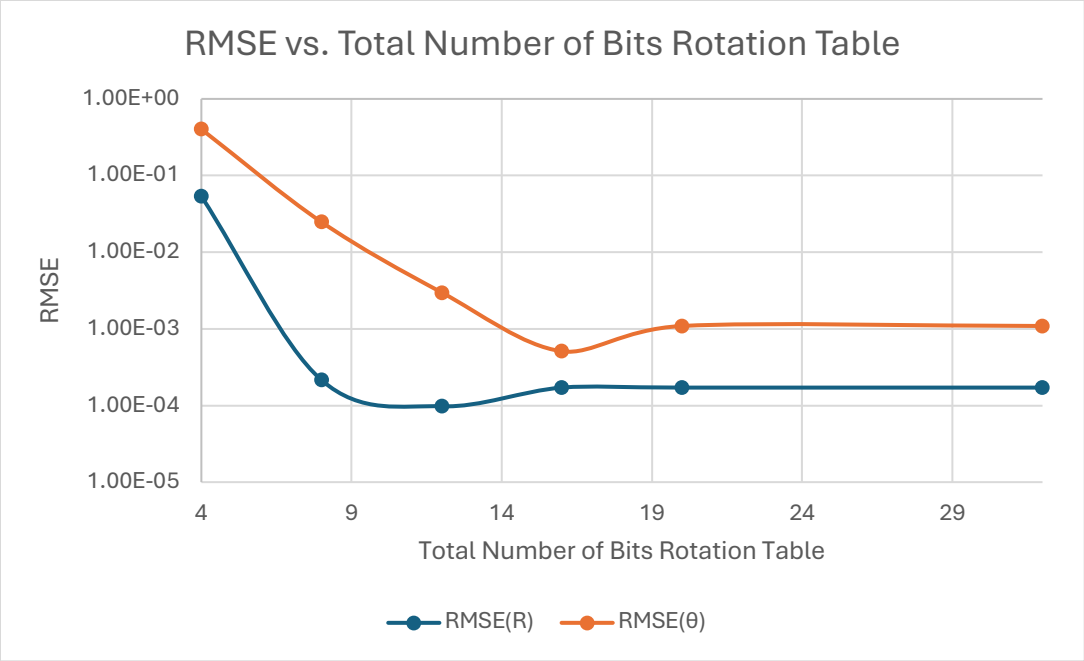




2c)

All variables: ap_fixed<16,3>. Only Kvalues and angles change total bit width (integer bits = 3).

Table bits	Est. clock (ns)	Latency (cycles)	II	Throughput (MHz)	BRAM	DSP	FF	LUT	RMSE(R)	RMSE(θ)
4	7.229	47	48	2.882	0	1	295	866	5.356249e-02	4.021737e-01
8	7.229	47	48	2.882	0	3	307	884	2.143911e-04	2.472714e-02
12	7.229	47	48	2.882	0	3	323	890	9.755724e-05	2.981615e-03
16	7.229	47	48	2.882	0	3	339	901	1.711696e-04	5.094576e-04
20	7.229	47	48	2.882	0	3	355	919	1.711696e-04	1.089674e-03
32	7.229	67	68	2.034	0	5	640	1060	1.711696e-04	1.089674e-03

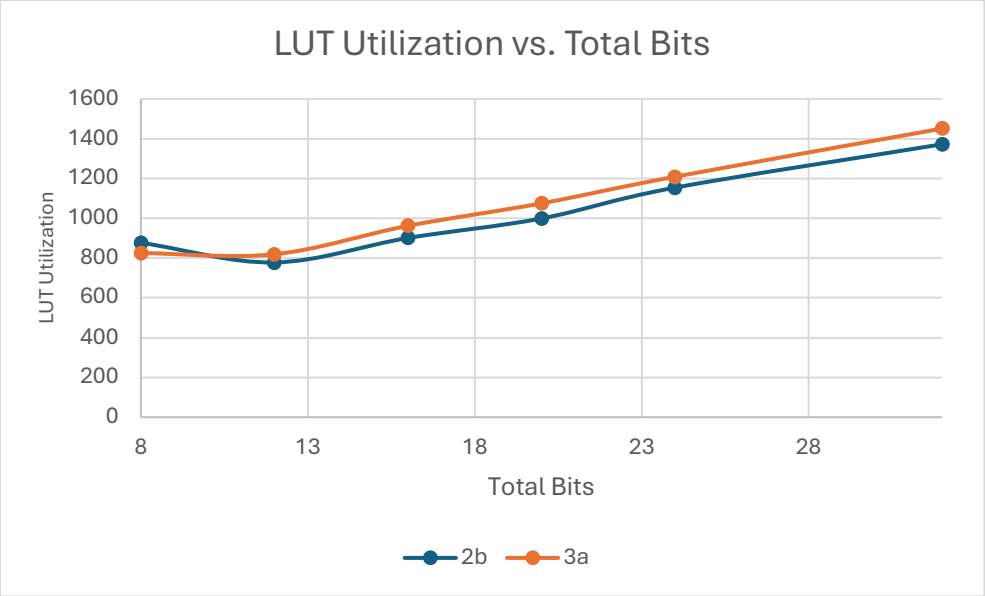


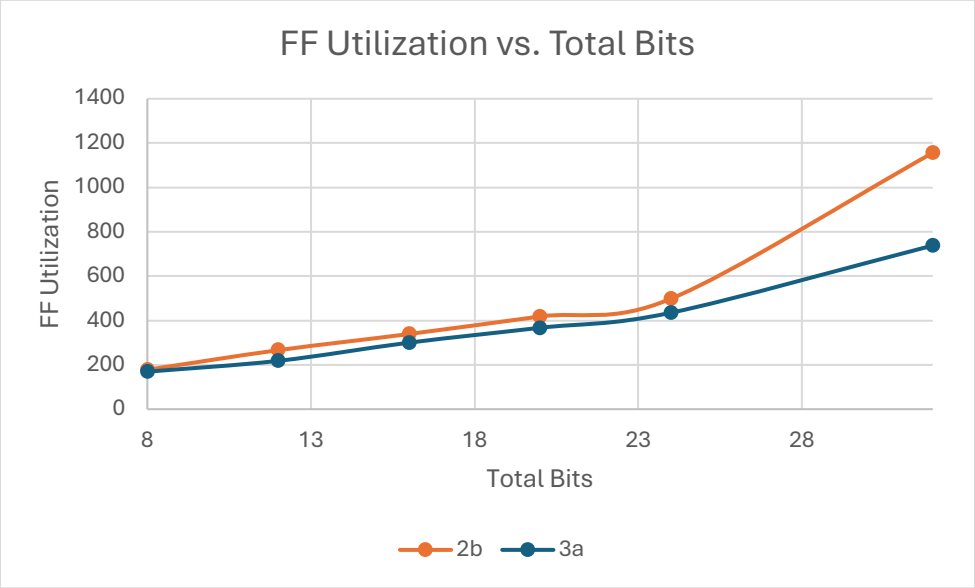
1_a)

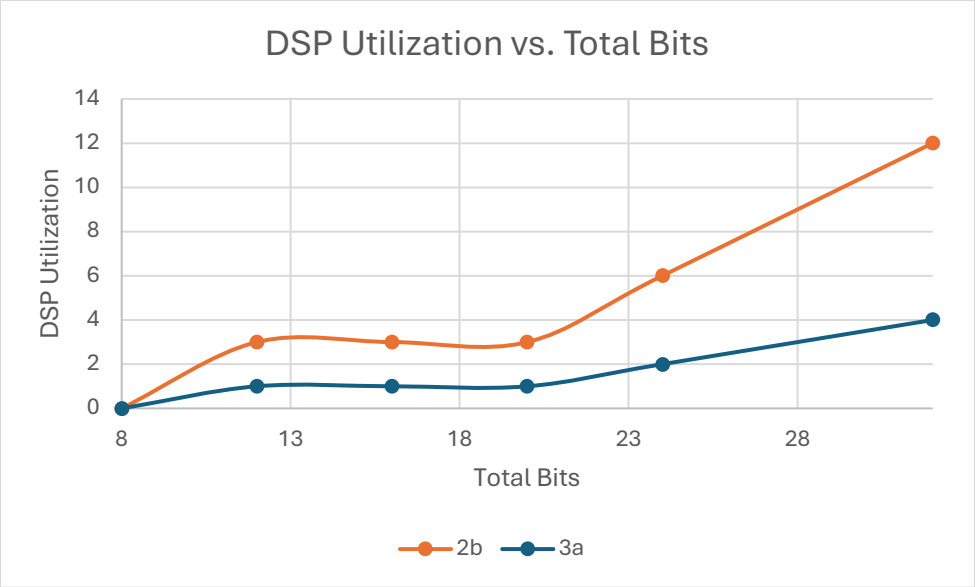
Question 3. Using addition and bit shifting instead of multiplication and division reduces hardware complexity and FPGA resource usage. However, CORDIC requires multiple iterations to achieve higher accuracy, which increases latency and reduces throughput.

3a.

Total bits	Est. clock (ns)	Latency (cycles)	Interval (cycles)	Throughput (MHz)	BRAM	DSP	FF	LUT	RMSE(R)	RMSE(θ)
8	7.274	46	47	2.925	0	0	169	825	2.088e-01	2.078e-02
12	6.704	46	47	3.174	0	1	218	819	7.627e-03	2.524e-03
16	7.229	45	46	3.007	0	1	300	962	1.712e-04	5.095e-04
20	6.814	45	46	3.190	0	1	367	1075	1.534e-04	1.375e-05
24	6.918	45	46	3.142	0	2	435	1209	1.414e-04	1.161e-06
32	7.184	46	47	2.962	0	4	738	1451	1.407e-04	6.172e-07


LUT Utilization vs. Total Bits
1600
1400
1200
1000
800
600
400
200
0
LUT Utilization
8
13
18
23
28
Total Bits
2b
3a


FF Utilization vs. Total Bits
1400
1200
1000
800
600
400
200
0
FF Utilization
8
13
18
23
28
Total Bits
2b
3a



Question 4.

4a) The input bit width determines the number of LUT entries, while the output bit width determines the size of each entry. Increasing the input bits increases memory exponentially, while increasing the output bits increases memory linearly. Since there are two inputs and two outputs, each with W bits, the total LUT size is

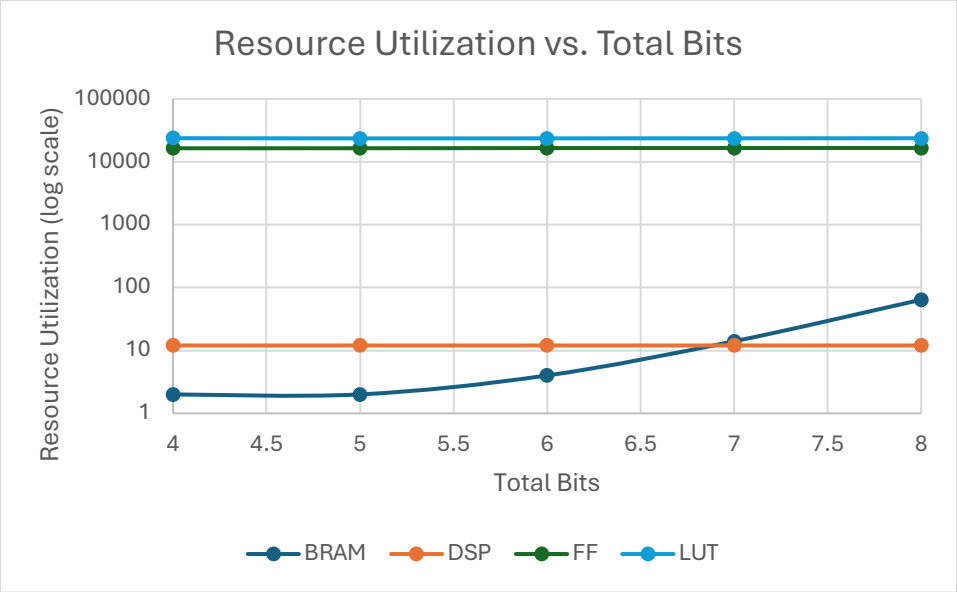
$$\text{LUT Size (bits)} = 2^{2W} \times 2W.$$

4b)

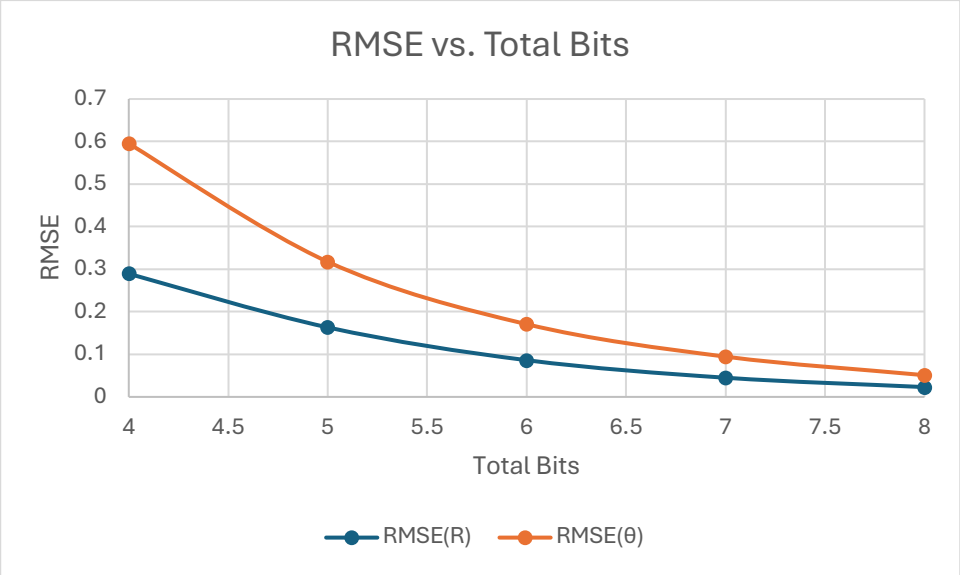
Resource usage, latency, throughput, and RMSE versus total bit width (3 integer bits for all data types).

Total bits	BRAM	DSP	FF	LUT	Est. clock (ns)	Latency (cycles) (min-max)	Interval (cycles) (min-max)	Throughput (MHz)	RMSE(R)	RMSE(θ)
4	2	12	16,400	23,721	7.256	3–332	4–333	0.414	0.289403886	0.595246792
5	2	12	16,432	23,558	7.256	3–1,100	4–1,101	0.125	0.163360938	0.317522198
6	4	12	16,522	23,597	7.256	3–4,172	4–4,173	0.033	0.0862983093	0.17088145
7	14	12	16,546	23,633	7.256	3–16,460	4–16,461	0.008	0.0450451337	0.0945838913
8	64	12	16,574	23,679	7.256	3–65,612	4–65,613	0.002	0.0230940841	0.0510457344

4c)



4d)



4e) CORDIC uses fewer hardware resources and scales better with higher bit widths but requires multiple iterations and increasing latency. The LUT implementation provides faster calculations through direct lookups but requires significantly more memory as the bit width increases. Overall, CORDIC is better for saving resources, while LUTs are better for fast lookups when sufficient memory is available.

Github Repo:

https://github.com/Math140Instructor/wes237c/tree/main/project2

Demo:
https://drive.google.com/file/d/140KmzKAiCQhyt7QbixA_NSrgLRQSJohS/view?usp=sharing